UE24CS341A SOFTWARE ENGINEERING · PES UNIVERSITY · JACKFRUIT MINI-PROJECT

Project Test Plan

Team 14 · Project 39: Dehallucination Add-On Module ("Dehallucinator")

Team member	Role
Sai Nikitha Perumalla (PES2UG24AM142)	Role A – Claim Extractor; Scrum Master
Shreya Raghuraj (PES2UG24AM154)	Role B – Evidence Retriever
Ralph D'souza (PES2UG24AM131)	Role C – Verifier & Scorer, testing, SonarQube
Satwik (PES2UG24AM148)	Role D – Corrector, API, UI, Jenkins, Docker

Repository: github.com/SoftwareEngineering-Runtime/dehallucinator · Jira project: DH

Test results: automated run of 07 Nov 2026

Date: 07 Nov 2026

Test Plan and Test Cases

Project 39: Dehallucination Add-On Module ("Dehallucinator")

Owner: Ralph D'souza (Role C). Each member writes the test cases for their own use case.

1. Scope

This document outlines the testing strategy for the Dehallucination Add-On Module. It covers unit testing for individual modules (Claim Extractor, Evidence Retriever, Verifier & Scorer, Corrector & Integrator), integration testing for component handoffs, system testing for the end-to-end API, and security testing.

2. Test Levels

- **Unit testing:** isolated testing of individual functions using mocks and fakes (no internet or model downloads permitted).
- **Integration testing:** testing the data flow between connected modules (e.g., Extractor → Retriever → Verifier).
- **System testing:** end-to-end testing of the complete `/check` API and pipeline using the 10 predefined sample answers.
- **Security testing:** verifying boundary limits, malformed inputs, and checking for vulnerabilities using bandit and pip-audit.

3. Environment

- **Language & tools:** Python 3.11, pytest, pytest-cov, ruff
- **Infrastructure:** Jenkins CI/CD pipeline, Docker, SonarQube Cloud
- **Dependencies:** spaCy (`en_core_web_sm`), transformers (`nli-deberta-v3-xsmall`), SQLite (caching)

4. Entry and Exit Criteria

- **Entry criteria:** code must be pushed via a pull request to `main`, pass the Jenkins pipeline linting (ruff), and pass the SonarQube Quality Gate.
- **Exit criteria:** all modules must achieve ≥ 80% line and branch coverage. All test cases listed in this document must pass, with 0 critical security vulnerabilities.

5. Traceability Matrix

Requirement	Test cases
FR-1.1	TC-1-01, TC-1-02
FR-1.2	TC-1-03
FR-1.3	TC-1-04, TC-1-05, TC-1-06
FR-1.4	TC-1-07
FR-1.5	TC-1-08
FR-1.6	TC-1-09

Requirement	Test cases
FR-1.7	TC-1-10
FR-2.1 to FR-2.8 (Role B)	TC-2-01 to TC-2-10 (see UC-2 traceability in Section 6)
FR-3.1	TC-3-07
FR-3.2	TC-3-06
FR-3.3	TC-3-02, TC-3-03, TC-3-05
FR-3.4	TC-3-04
FR-3.5	TC-3-10, TC-3-11
FR-3.6	TC-3-08, TC-3-09
FR-4.1	TC-4-01, TC-4-02, TC-4-03, TC-4-04, TC-4-05
FR-4.2	Unit tests in `tests/unit/test_pipeline.py`; integration tests in `tests/integration/test_pipeline.py` (DH-197)
FR-4.3	TC-4-10
FR-4.4	TC-4-06, TC-4-07, TC-4-08, TC-4-09
FR-4.5	UI tests in `tests/unit/test_ui.py`; system tests on the 10 samples (DH-198)
FR-4.6	UI tests in `tests/unit/test_ui.py` (colours, escaping); system tests (DH-198)

6. Test Cases

Each use case has 8–10 test cases, with at least 2 error or edge cases. "Automated test" names the pytest function that checks the case. The Actual and Pass/Fail columns come from the automated test run on 07 Nov 2026 (154 tests passed, 0 failed; the 2 system/security placeholder tests are skipped until Sprint 3).

UC-1: Extract factual claims (Owner: Sai Nikitha Perumalla, Role A)

Automated tests: `tests/unit/test_extractor.py`

TC ID	FR	Type	Description	Input	Expected result	Automated test	Actual	Pass/ Fail
TC-1-01	FR-1.1	Error	Empty input is rejected	`""` and `" \n\t "`	`ValueError("Input text is empty")`	`test_tc_1_01_empty_input_is_rejected`	As expected (2 checks passed)	**Pass**
TC-1-02	FR-1.1	Boundary	Input over the limit is rejected	5001 characters (and exactly 5000)	5001 → `ValueError` mentioning 5000; 5000 is accepted	`test_tc_1_02_input_over_limit_is_rejected`, `test_input_exactly_at_limit_is_accepted`	As expected (2 checks passed)	**Pass**
TC-1-03	FR-1.2	Normal	Text is split into sentences	"Paris is in France. It is a large city."	2 claims ("It" becomes "Paris" by FR-1.5); "Dr." / "U.S." do not cause false splits	`test_tc_1_03_splits_two_sentences`, `test_abbreviations_do_not_cause_false_splits`	As expected (2 checks passed)	**Pass**
TC-1-04	FR-1.3	Normal	Questions are dropped	"Paris is the capital of France. Is it the largest city?"	Only the first sentence is a claim	`test_tc_1_04_questions_are_dropped`	As expected (1 check passed)	**Pass**

TC ID	FR	Type	Description	Input	Expected result	Automated test	Actual	Pass/ Fail
TC-1-05	FR-1.3	Normal	Opinion sentences are dropped	"I think Paris is the most beautiful city.", "Maybe the tower was built in 1889." and 3 more	No claims; markers match whole words in any case ("shoulder" is kept)	`test_tc_1_05_opinion_sentences_are_dropped`, `test_opinion_markers_ignore_case`, `test_opinion_marker_must_be_a_whole_word`	As expected (7 checks passed)	**Pass**
TC-1-06	FR-1.3	Boundary	Short and first-person sentences are dropped	"Paris is big." (3 words), "Paris is very big." (4 words), "I visited Paris in 2019."	3-word sentence dropped, 4-word sentence kept, I/We sentence dropped	`test_tc_1_06_short_sentences_are_dropped`, `test_first_person_sentences_are_dropped`	As expected (3 checks passed)	**Pass**
TC-1-07	FR-1.4	Normal	Compound sentence is split	"Einstein was born in 1879, and he won the Nobel Prize in 1921."	2 claims; "Salt and pepper are spices." stays 1 claim	`test_tc_1_07_compound_sentence_is_split`, `test_list_with_and_is_not_split`, `test_part_without_its_own_subject_is_not_split`	As expected (3 checks passed)	**Pass**
TC-1-08	FR-1.5	Normal	Leading pronoun is replaced	"The Eiffel Tower is in Paris. It is 330 m tall."	Second claim is "The Eiffel Tower is 330 m tall."	`test_tc_1_08_leading_pronoun_is_replaced`, `test_person_and_possessive_pronouns_use_the_last_person`, `test_it_and_he_pick_different_entities`, `test_pronoun_without_earlier_entity_is_kept`	As expected (4 checks passed)	**Pass**
TC-1-09	FR-1.6	Edge	Duplicate claims are removed	"Paris is in France. paris is in France!"	1 claim (first occurrence kept)	`test_tc_1_09_duplicate_claims_are_removed`, `test_ids_have_no_gaps_after_duplicates_are_removed`, `test_duplicate_found_after_pronoun_replacement`, `test_normalise_ignores_case_punctuation_and_spaces`	As expected (4 checks passed)	**Pass**
TC-1-10	FR-1.7	Normal	Claims are numbered without gaps	"Mars is the fourth planet. Is that right? Jupiter is the largest planet."	ids `[1, 2]`, sentence_index `[0, 2]`; all-opinion text returns `[]`	`test_ids_have_no_gaps_and_keep_original_sentence_index`, `test_all_sentences_non_factual_returns_empty_list`	As expected (2 checks passed)	**Pass**

Error and edge cases: TC-1-01, TC-1-02, TC-1-06, TC-1-09.

UC-2: Retrieve evidence for a claim (Owner: Shreya Raghuraj, Role B)

Automated tests: `tests/unit/test_retriever.py`, `test_ranker.py`, `test_cache.py`, `test_retries_and_cache.py`, `test_retrieve_evidence.py`

TC ID	FR	Type	Description	Input	Expected result	Automated test	Actual	Pass/ Fail
TC-2-01	FR-2.1	Normal	Entities come first in query	"The famous bridge was designed by Gustave Eiffel in Paris."	Query contains Gustave, Paris and bridge; entity words appear before noun-chunk words	`test_tc_2_01_entities_come_before_noun_chunks`	As expected (1 check passed)	**Pass**

TC ID	FR	Type	Description	Input	Expected result	Automated test	Actual	Pass/ Fail
TC-2-02	FR-2.1	Normal	Stop words removed, max 8 words	Long claim with more than 8 content words	At most 8 words; no stop words	`test_tc_2_02_stop_words_removed_and_capped_at_8_words`	As expected (1 check passed)	**Pass**
TC-2-03	FR-2.2	Normal	Wikipedia search request	Query "Eiffel Tower"	Request uses `srlimit=3` and sends a User-Agent header; returns page titles	`test_tc_2_03_search_uses_limit_and_user_agent`	As expected (1 check passed)	**Pass**
TC-2-04	FR-2.3	Normal	Page text fetch	Title "Eiffel Tower", 10000-char extract	Text cut to 6000 chars; URL is `https://en.wikipedia.org/wiki/Eiffel_Tower`	`test_tc_2_04_fetch_truncates_and_builds_url`	As expected (1 check passed)	**Pass**
TC-2-05	FR-2.5	Normal	Snippets ranked by relevance	Claim "The Eiffel Tower was completed in 1889"	At most 3 results, best first, relevance between 0 and 1 (cosine score); irrelevant page excluded	`test_tc_2_05_ranked_by_relevance_across_sources`	As expected (1 check passed)	**Pass**
TC-2-06	FR-2.4	Normal	KB loaded once at start-up	Start retriever, then retrieve for two claims	Every `data/kb/*.txt` is loaded as a page with source "local_kb"; files are read once, not per claim	`test_offline_uses_local_kb`	As expected (1 check passed); manual run: 15 KB files loaded as local_kb pages, second call served from cache (hits=1, misses=1)	**Pass**
TC-2-07	FR-2.6	Normal	Wikipedia page cached and reused	Same page requested twice; then again with a cache entry 8 days old	Second request makes no HTTP call; the 8-day-old entry is fetched again	`test_hit_returns_stored_page`, `test_expired_entry_is_ignored`, `test_fetch_page_text_uses_cache_without_network`, `test_fetch_page_text_stores_result_in_cache`	As expected (4 checks passed)	**Pass**
TC-2-08	FR-2.7, NFR-2	Error	Wikipedia down: KB fallback	Claim about a topic covered by the KB	2 retries with 5 s timeout, warning logged, Evidence with source "local_kb" returned, no exception	`test_retries_then_succeeds`, `test_gives_up_after_retries`, `test_network_down_falls_back_to_kb_only`	As expected (3 checks passed)	**Pass**
TC-2-09	FR-2.8	Error	No evidence found	Claim "Zxqv blorf quux." and an empty claim list	No Evidence for the nonsense claim; empty list for an empty claim list	`test_no_results_and_no_kb_gives_empty_list`, `test_empty_claims_list`	As expected (2 checks passed)	**Pass**
TC-2-10	NFR-3	Error	SQL-like input does not break cache	Title or claim containing `'; DROP TABLE pages; --`	No exception; cache table intact; later lookups still work	`test_sql_like_title_is_safe`	As expected (1 check passed)	**Pass**

Also automated: `test_empty_claim_returns_empty_query` (FR-2.1, empty claim) and `test_empty_inputs_return_empty_list` (FR-2.5, empty input).

Error cases: TC-2-08, TC-2-09, TC-2-10.

UC-2 traceability

Requirement	Description	Test cases
FR-2.1	Build search query per claim	TC-2-01, TC-2-02
FR-2.2	Search Wikipedia, top 3, User-Agent	TC-2-03
FR-2.3	Fetch page text, first 6000 chars	TC-2-04
FR-2.4	Load local KB once at start-up	TC-2-06
FR-2.5	Rank snippets by TF-IDF cosine, top 3	TC-2-05
FR-2.6	Cache Wikipedia pages in SQLite, 7 days	TC-2-07
FR-2.7	No crash on network failure, retry twice, KB fallback	TC-2-08
FR-2.8	No Evidence when nothing found	TC-2-09
NFR-2	Works when Wikipedia is down	TC-2-08
NFR-3	SQL-like input must not break the cache	TC-2-10

UC-3: Verify a claim against its evidence (Owner: Ralph D'souza, Role C)

Automated tests: `tests/unit/test_verifier.py`

TC ID	FR	Type	Description	Input	Expected result	Automated test	Actual	Pass/ Fail
TC-3-02	FR-3.3	Positive	High entailment supports the claim	Claim: “Paris is in France.” Evidence: “Paris is the capital of France.”	SUPPORTED; score 0.9; matching evidence returned	`test_tc_3_02_high_entailment_is_supported`	As expected (1 check passed)	**Pass**
TC-3-03	FR-3.3	Positive	High contradiction contradicts the claim	Claim: “Paris is in Italy.” Evidence: “Paris is the capital of France.”	CONTRADICTED; score 0.9; matching evidence returned	`test_tc_3_03_high_contradiction_is_contradicted`	As expected (1 check passed)	**Pass**
TC-3-04	FR-3.4	Negative	Claim has no evidence	Claim: “Unknown fact.” No evidence supplied	NOT_ENOUGH_INFO; score 0.0; no best evidence; model not called	`test_tc_3_04_no_evidence_is_not_enough_info`	As expected (1 check passed)	**Pass**
TC-3-05	FR-3.3	Boundary	Evidence is uncertain	Claim: “The fact is true.” Evidence: “Some unrelated information.”	NOT_ENOUGH_INFO; score 0.7; best evidence returned	`test_tc_3_05_uncertain_evidence_is_not_enough_info`	As expected (1 check passed)	**Pass**
TC-3-06	FR-3.2	Positive	Only evidence matching the claim is used	One evidence item for claim 2 and one for claim 1	SUPPORTED; matching evidence selected; predictor called once with matching pair	`test_tc_3_06_only_matching_claim_evidence_is_used`	As expected (1 check passed)	**Pass**
TC-3-07	FR-3.1	Positive	Model and tokenizer are loaded on demand	Model and tokenizer initially unset	Both loaded using configured NLI model	`test_tc_3_07_model_is_loaded_lazily`	As expected (1 check passed)	**Pass**
TC-3-08	FR-3.6	Boundary	Empty verdict list returns the default trust score	Empty list `[]`	Trust score is 100.0	`test_trust_score_empty_list`	As expected (1 check passed)	**Pass**

TC ID	FR	Type	Description	Input	Expected result	Automated test	Actual	Pass/ Fail
TC-3-09	FR-3.6	Normal	Trust score averages mixed verdicts	One SUPPORTED, one NOT_ENOUGH_INFO, one CONTRADICTED verdict	Trust score is 50.0	`test_trust_score_mixed_verdicts`	As expected (1 check passed)	**Pass**
TC-3-10	FR-3.5	Negative	A number mismatch for the same named entity forces contradiction	Claim: “Einstein was born in 1879.” Evidence: “Einstein was born in 1899.”	CONTRADICTED; score is `max(max_contradiction, 0.75)` ; matching evidence returned	`test_tc_3_10_year_mismatch_forces_contradicted`	As expected (1 check passed)	**Pass**
TC-3-11	FR-3.5	Positive	Matching numbers do not trigger the mismatch rule	Claim: “Einstein was born in 1879.” Evidence: “Einstein was born in 1879.”	SUPPORTED; score 0.9; matching evidence returned	`test_tc_3_11_matching_numbers_remain_supported`	As expected (1 check passed)	**Pass**

UC-4: Check and correct a chatbot answer (Owner: Satwik, Role D)

Automated tests: `tests/unit/test_corrector.py`, `tests/unit/test_api.py`, `tests/unit/test_guard.py`

TC ID	FR	Type	Description	Input	Expected result	Automated test	Actual	Pass/ Fail
TC-4-01	FR-4.1	Normal	Contradicted claim is replaced with the correction and source	“The Eiffel Tower was completed in 1899.” CONTRADICTED, evidence “… completed in 1889.”	“The Eiffel Tower was completed in 1889. [corrected, source:]”; “1899” no longer appears	`test_tc_4_01_contradicted_claim_is_replaced_with_correction_and_source`	As expected (1 check passed)	**Pass**
TC-4-02	FR-4.1	Normal	Unverifiable claim is flagged	“The Eiffel Tower has 1,710 steps.” NOT_ENOUGH_INFO	Claim kept, followed by `[unverified]`	`test_tc_4_02_not_enough_info_claim_is_tagged_unverified`	As expected (1 check passed)	**Pass**
TC-4-03	FR-4.1	Normal	Supported claim is tagged with its source	“The Eiffel Tower is located in Paris.” SUPPORTED with evidence	Claim kept, followed by `[source: <url>]`	`test_tc_4_03_supported_claim_is_tagged_with_its_source`	As expected (1 check passed)	**Pass**
TC-4-04	FR-4.1	Normal	Text that is not a claim is left unchanged	“Hello! … I hope this helps. …” with one CONTRADICTED and one SUPPORTED claim	Greeting and closing sentence unchanged; only the two claims are rewritten or tagged	`test_tc_4_04_mixed_answer_keeps_non_claim_text_unchanged`	As expected (1 check passed)	**Pass**
TC-4-05	FR-4.1	Edge	Contradicted claim without evidence, and claim without a verdict	“Mars has two moons.” CONTRADICTED with no evidence; “Mars is the largest planet.” with no verdict	Both claims kept and tagged `[unverified]`	`test_tc_4_05_contradicted_without_evidence_and_missing_verdict_are_unverified`	As expected (1 check passed)	**Pass**
TC-4-06	FR-4.4	Normal	POST /check returns the checked answer	`POST /check {"text": "The Eiffel Tower is located in Paris."}`	200; body is the `CheckedAnswer` JSON	`test_tc_4_06_check_returns_checked_answer`	As expected (1 check passed)	**Pass**
TC-4-07	FR-4.4	Normal	Health check	`GET /health`	200; `{"status": "ok"}`	`test_tc_4_07_health_returns_ok`	As expected (1 check passed)	**Pass**

TC ID	FR	Type	Description	Input	Expected result	Automated test	Actual	Pass/ Fail
TC-4-08	FR-4.4	Error	Invalid input is rejected	Missing, empty, blank, wrong-type (42), 5001-character and wrong-field bodies; malformed JSON	422 for each; the pipeline is not called; exactly 5000 characters is accepted	`test_tc_4_08_invalid_input_returns_422`, `test_tc_4_08_malformed_json_returns_422`, `test_exactly_5000_characters_is_accepted`	As expected (8 checks passed)	**Pass**
TC-4-09	FR-4.4	Error	Unexpected error does not leak details	Pipeline raises `RuntimeError("secret internal detail")`	500; `{"detail": "Internal server error"}`; no message or traceback in the response; error logged on the server	`test_tc_4_09_unexpected_error_returns_500_without_stack_trace`	As expected (1 check passed)	**Pass**
TC-4-10	FR-4.3	Normal	Decorator turns a chatbot function into a checked one	`@dehallucinate` on a function returning “The Eiffel Tower is located in Paris.”; also functions returning None, 42, a list, bytes	Returns a `CheckedAnswer` for the string; name and docstring kept; non-str return raises `TypeError`	`test_tc_4_10_decorated_function_returns_checked_answer`, `test_name_and_docstring_are_kept`, `test_non_string_return_raises_type_error`	As expected (6 checks passed)	**Pass**

Error and edge cases: TC-4-05, TC-4-08, TC-4-09.

7. Test Run Summary (07 Nov 2026)

Use case	Test cases	Passed	Failed
UC-1: Extract factual claims	10	10	0
UC-2: Retrieve evidence for a claim	10	10	0
UC-3: Verify a claim against its evidence	10	10	0
UC-4: Check and correct a chatbot answer	10	10	0
Total	**40**	**40**	**0**

Full suite: 154 passed, 2 skipped (system and security placeholders, Sprint 3), 92% line + branch coverage of `src/`, ruff lint clean, SonarCloud quality gate passed.